NLP-to-SQL POC Design: Schema Prep and LLM Integration (Vertica, Zero-Cost Stack)

🌐 Objective:

Build a low-cost NLP-to-SQL solution using LLMs (e.g., LLaMA via Ollama), Vertica database, and zero-cost tools.

✅ Components Overview:

1. **LLM (Ollama + LLaMA)**
   - Converts natural language questions into SQL queries.
   - Does **not** execute SQL; only generates it.
2. **Execution Module**
   - Takes generated SQL.
   - Executes it on Vertica.
   - Returns result to user.
3. **Schema Preparation (Schema Prep)**
   - Prepares your database schema for use with the LLM.
   - Acts like a lightweight knowledge base (KB).

🔧 Schema Prep Pipeline (for Vertica)

Step	Description
1. Extract Schema	Pull table names, column names, data types, and optional descriptions from Vertica.
2. Format as Text	Convert into natural-language text format (LLM-friendly).
3. Chunking	Break schema into small logical pieces (table-wise or token-limit-wise).
4. Embeddings	Use a local embedding model (e.g., HuggingFace or Open Source) to convert text to vectors.
5. Store in Vector DB	Save embeddings in a vector DB (like FAISS or Chroma) for similarity search.

🔍 Usage at Query Time

1. Embed the user query.
2. Perform similarity search on the vector DB.
3. Retrieve relevant schema chunks.
4. Provide both user query + relevant schema to LLM.
5. LLM generates SQL.
6. Execute using Vertica client.

🚀 Tools to Use (Zero-Cost Focus)

Need	Suggested Tools
LLM	Ollama + LLaMA
Embedding Model	BGE-Small, MiniLM, or other open-source models
Vector DB	FAISS (local) or Chroma
Frontend (Optional)	Streamlit, Flask
Database	Vertica (existing)

🔎 Example Schema Format (for Embedding)

```
Table: orders
Description: Contains details of customer orders
Columns:
- order_id (int): Unique ID for the order
- customer_id (int): ID of the customer
- order_date (date): Date the order was placed
- amount (float): Total order value
```

📊 When to Use Each:

Term	When It's Used
Embeddings	When you need to semantically compare user queries and schema content
Chunking	When schema text is large (beyond token limit) or complex
Vector DB	To store and search embeddings efficiently
Schema Prep	Before deployment, to make schema LLM-friendly

NLP-to-SQL POC Design: Schema Prep and LLM Integration (Vertica, Zero-Cost Stack)

🔧 Overview:

- You're building an **NLP-to-SQL POC** using **LLMs** (Ollama + LLaMA) on your **own datacenter** (zero-cost approach).
- The database is **Vertica**.
- Goal: Convert natural language questions into SQL queries using LLMs.

✅ Key Concepts Explained for Your POC:

1. Embeddings

- **What**: Numeric representations of text that capture meaning.
- **Why**: Helps LLMs retrieve relevant schema/table info based on semantic similarity.
- **Example in Your POC**:
  - Convert: "*orders table has order_id, customer_id, total_amount...*" into a vector.
  - Convert user query: "*how much did customer 1001 spend?*" into a vector.
  - Find closest match in schema vectors.

2. Chunking

- **What**: Splitting large schema text into smaller parts for embedding.
- **Why**: LLMs or embedding models have token limits (e.g., 512 or 1024 tokens).
- **Example in Your POC**:
  - Chunk per table:

3. **Vector DB**

- **What**: A database that stores embeddings (vectors) and performs similarity search.
- **Why**: Allows finding relevant schema chunks based on user question.
- **Examples**: FAISS (zero-cost), Chroma, Milvus.
- **Example in Your POC**:
  - Store chunk vectors.
  - When user asks a question, convert to vector, retrieve top-k relevant schema chunks.

4. **KB (Knowledge Base)**

- **What**: A structured source of truth (here: your DB schema info).
- **Why**: Helps LLMs understand your data structure.
- **Example in Your POC**:
  - KB is made from schema text like:
    "The `orders` table tracks purchases with fields like `order_id`, `customer_id`, etc."
  - Embedded and stored in the vector DB.

🔄 Schema Prep Workflow:

1. **Extract Vertica Schema Info**:
   - Use SQL to query metadata tables.
2. **Format as Natural Language Text**:
3. **Chunk if necessary** (e.g., per table or block of columns).
4. **Generate Embeddings** (using LLaMA-compatible model).
5. **Store in Vector DB (like FAISS)**.
6. **At runtime**:
   - User asks question.
   - Convert question to embedding.
   - Search vector DB.
   - Retrieve relevant schema chunks.
   - Feed to LLM to generate SQL.
7. **Execute SQL in separate module (not LLM)**.

Let me know if you want to add:

- Vertica schema extraction SQL script
- Diagram of the full pipeline
- LLM prompt example
- Evaluation strategy

📁 Project Structure (Python + FAISS + Embeddings + Vertica + LLM Prompting)

```
nlp_sql_poc/
|
```

```
├── 📁 data/
│ ├── schema_raw/ # Raw text versions of your DB table schemas
│ └── user_queries.txt # Optional: sample user queries for testing
│
├── 📁 embeddings/
│ ├── generate_embeddings.py # Chunk + embed schema and store vectors
│ ├── schema_chunks.pkl # Serialized schema chunks
│ └── faiss_index.index # FAISS index file
│
├── 📁 llm/
│ ├── prompt_builder.py # Dynamically build prompt from chunks
│ ├── ollama_llm.py # LLaMA call via Ollama (local inference)
│
├── 📁 query_handler/
│ ├── vector_search.py # FAISS vector search
│ ├── validate_sql.py # Basic SQL validation / syntax checker
│ ├── execute_sql.py # Send query to Vertica & fetch results
│
├── 📁 utils/
│ ├── chunker.py # Text chunking logic
│ ├── db_connector.py # Vertica DB connection utilities
│ └── config.py # Configs (LLM model, DB, paths, etc.)
│
├── main.py # Entrypoint to run full pipeline
├── requirements.txt # Dependencies
└── README.md # Project overview & instructions
```

🔁 End-to-End Flow

1. **Prepare schema** → manually or script export → store in `data/schema_raw`
2. **Chunk + Embed** schemas → save in FAISS index (`generate_` embeddings.py)
3. **User Query** → embed → vector search → get relevant schema chunks
4. **Prompt** → build prompt from query + schema
5. **LLM** → generate SQL from prompt
6. **Validation** → basic SQL check
7. **Execute SQL** on Vertica → return results

```
from embeddings.generate_embeddings import load_embeddings
from query_handler.vector_search import get_relevant_chunks
from llm.prompt_builder import build_prompt
from llm.ollama_llm import query_llm
from query_handler.validate_sql import is_valid_sql
from query_handler.execute_sql import run_query

user_query = "Get 10 days of failed server paths"
index, schema_chunks = load_embeddings()
relevant_chunks = get_relevant_chunks(user_query, index, schema_chunks)
prompt = build_prompt(user_query, relevant_chunks)
```

```
sql = query_llm(prompt)

if is_valid_sql(sql):
result = run_query(sql)
print("Query:\n", sql)
print("Result:\n", result)
else:
print("Invalid SQL generated.")
```

Sample `requirements.txt`

```
sentence-transformers
faiss-cpu
transformers
numpy
vertica-python
requests
```

✅ Recommended GitHub Branching Strategy

🔹 1. **`main`** (or **`prod`**)

- **Stable production code only**
- Deployed or ready-to-demo version
- Reviewed, tested, and validated

🔹 2. **`dev`**

- Ongoing development branch
- All new features/bugfixes merged here first
- Integration and testing happens here

📦 Feature Branches (`feature/`)

Use for new isolated features:

- `feature/vector-search`
- `feature/llm-prompting`
- `feature/vertica-integration`

➡️ Branch from `dev`, PR back to `dev`

🐞 Bugfix Branches (`bugfix/`)

For fixing issues or errors:

- `bugfix/faiss-index-error`
- `bugfix/sql-validation`

➡️ Branch from `dev`, PR back to `dev`

🔧 Experiment Branches (`exp/`)

For risky or experimental ideas (prompt tuning, different LLMs, etc.):

- `exp/llama3-prompt-style2`
- `exp/prompt-cot-llm`

➡️ PR back to `dev` if it works out

🔁 Hotfix Branches (`hotfix/`)

Emergency patches on `main`:

- `hotfix/db-connection-timeout`

➡️ Branch from `main`, PR to both `main` and `dev`

🧪 Testing Branches (`test/`)

Optional: for testing stuff in isolation

- `test/sql-runner-cli`
- `test/load-testing`

⛓️ Sample Workflow

```
# Feature development
git checkout dev
git checkout -b feature/vector-search

# Commit work
git commit -m "Add FAISS vector search"
git push origin feature/vector-search

# Open PR to dev → test → merge
```

🗂️ Summary Table

Branch Type	Purpose	PR Target
`main`	Stable production/demo-ready	—
`dev`	Integration & active development	`main`
`feature/*`	New feature work	`dev`
`bugfix/*`	Fix non-prod bugs	`dev`
`hotfix/*`	Fix prod issues	`main`, `dev`
`exp/*`	Experimental ideas/tests	`dev` (if good)
`test/*`	Manual/integration test zones	`dev` (optional)